

CS571 Programming Languages

Spring 2026

1. Introduction



CS571 is NOT

- C/C++/Java/Python/... Programming
- Data structures (CS 575)
- Object-oriented programming (CS 540)
- Compiler construction (CS 572)



CS571

- Explores **fundamental principles and paradigms** of programming languages
- Studies features found in many different languages and examine how they work and how they interact with each other

Why CS571?

- Programming languages facilitate communication of ideas
 - Between people
 - Between human and computers
- This course explores the **fundamental principles** of programming languages, to facilitate communication in your future study/career



Why CS571?

- Programming languages is a powerful tool once you master the principles
- Designing new languages (or features) for building secure software is an active research area

Why CS571?

- Example: use **type systems** to build secure software

```
int i = 0;
boolean b = true;
b = i; //incompatible types!
      //int cannot be assigned to boolean
```

Why CS571?

- Example: use **type systems** to build secure software

```
secret s = 0;  
public p = 1;  
p = s; //type error!  
        //cannot assign secret value to public variable
```

Language Dimensions

- When encountering a new language, it is useful to locate it in the multi-dimensional language space
 - Syntax
 - Semantics
 - Runtime
 - Paradigms
 - Concurrency, etc.

Syntax

- Most boring/trivial aspect of programming languages
 - Line-oriented syntax? [make](#), [C pre-processor](#)
 - Is whitespace or indentation significant? [Python](#), [occam](#), [make](#)
 - Does the language have C-like syntax?
 - Is the syntax minimal? [Lisp](#), [Prolog](#), [Lua](#), [many embedded languages](#)
 - Is the syntax homoiconic? That is, does the language easily treat code as data? [Lisp](#), [Clojure](#), [Prolog](#)

Semantics

- The meaning of a programming language
 - Is the language type-safe? [Most modern languages are type-safe, C is not](#)
 - When are type checks done? [Compile time vs. runtime](#)
 - Does it support mutability? (Can the value of a "variable" be changed after initialization?)
 - What object-oriented model does it use? [Single inheritance like Java, Python; multiple inheritance like C++](#)
 - Does it support functions as first-class values? That is, is it possible to handle functions just like any other values: store them in a variable, pass them between functions, etc.
 - Is memory visible to the programmer? More specifically, are memory addresses visible via entities like pointer? [C/C++](#)

Runtime

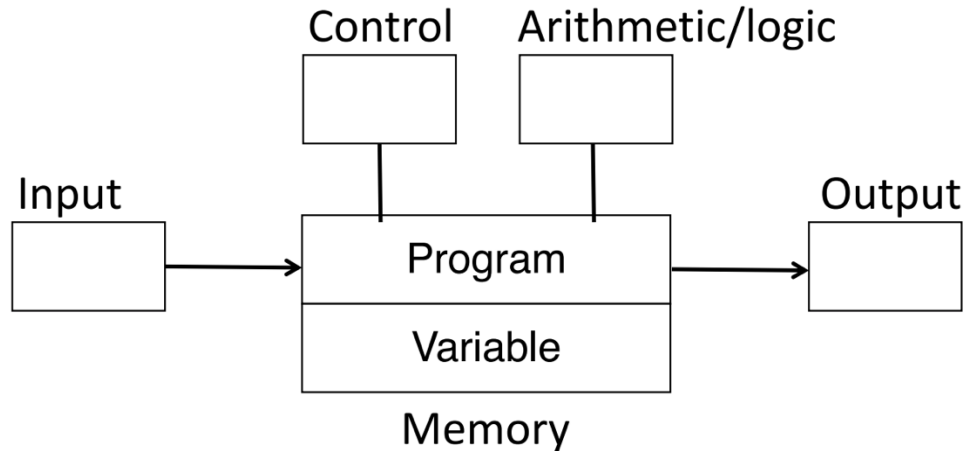
- Does it provide automatic memory management (garbage collection) or is the programmer responsible for memory management?
- Is it tied to a particular OS or architecture?
- Does it have extended library support?
- Is it possible to replace a running program with a new version *while it is running*?
- Are function arguments evaluated eagerly by the caller before call (most existing languages) or lazily by the callee as needed.
- Are there language features to support constructing code dynamically while a program is running?

Paradigms

- Most popular languages are **imperative** (includes object-oriented languages)
- Other programming languages are **declarative**, where the programmer use the programming language to tell the computer **what to compute rather than how to compute it**
 - Examples of declarative paradigms include:
 - Functional (**Scheme**)
 - Logic-based (**prolog**)
 - Constraint-based (**spreadsheets**)
 - Database (**SQL**)
- Many languages are mixed paradigm

Imperative Language

- Program is a series of updates to variables



The von Neumann-Eckert Model, 1945

Functional Language

- Program is a mathematical function

$$\text{output} = f(\text{input})$$

Logic Programming Language

- Program is a set of facts and rules

```
% Facts:  
father(mike, susan).  
mother(mary, susan).  
mother(jane, mary).  
  
% Rules:  
parent(X,Y) :- father(X,Y).  
parent(X,Y) :- mother(X,Y).  
grandparent(X,Y) :- parent(X,Z), parent(Z,Y).  
  
?- grandparent(jane, susan).  
true.
```

Example: Imperative

To compute the **gcd** of **a** and **b**, check if **a** and **b** are equal. If so, print one of them and stop. Otherwise, replace the larger one by their difference and repeat.

```
int gcd(int a, int b) {                                // C
    while (a != b) {
        if (a > b) a = a - b;
        else b = b - a;
    }
    return a;
}
```

Example: Functional

The **gcd** of **a** and **b** is defined to be:

1. **a** when **a** and **b** are equal,
2. the **gcd** of **b** and **a-b** when **a>b**, and
3. the **gcd** of **a** and **b-a** when **b>a**.

To compute the **gcd** of a pair of numbers, expand and simplify this definition until it terminates.

```
let rec gcd a b =                                     (* OCaml *)  
  if a = b then a  
  else if a > b then gcd b (a - b)  
  else gcd a (b - a)
```

Example: Logic

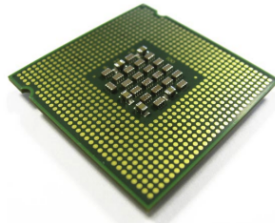
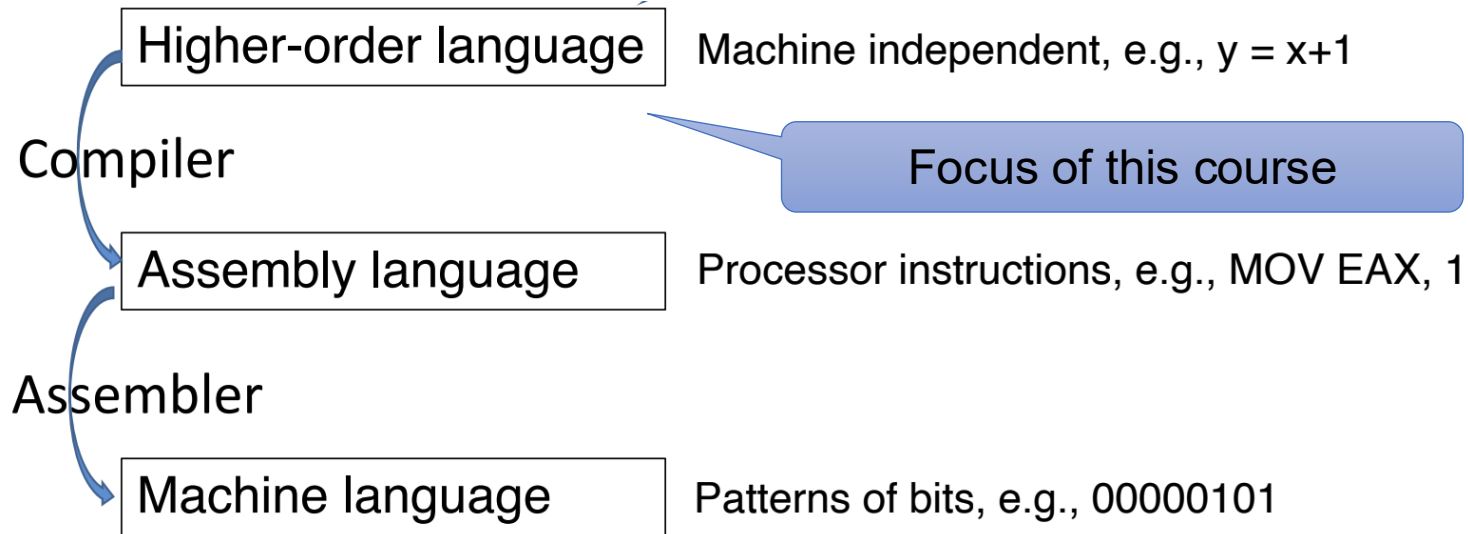
The proposition `gcd(a,b,g)` is true if:

1. `a`, `b`, `g` are all equal;
2. `a` is greater than `b`, and there exists a number `c` such that `c` is `a-b` and `gcd(c,b,g)` is true; or
3. `a` is less than `b`, and there exists a number `c` such that `c` is `b-a` and `gcd(c,a,g)` is true.

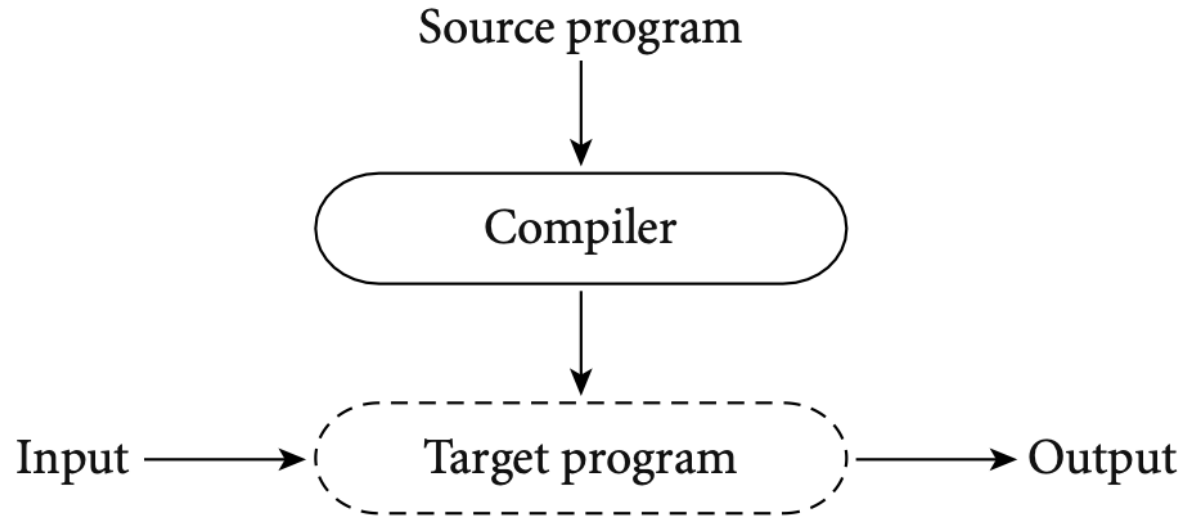
To compute the `gcd` of a pair of numbers, search for a number `g` (and various numbers `c`) for which these rules allow one to prove that `gcd(a,b,g)` is true.

```
gcd(A,B,G) :- A = B, G = A.                                % Prolog
gcd(A,B,G) :- A > B, C is A-B, gcd(C,B,G).
gcd(A,B,G) :- B > A, C is B-A, gcd(C,A,G).
```

Levels



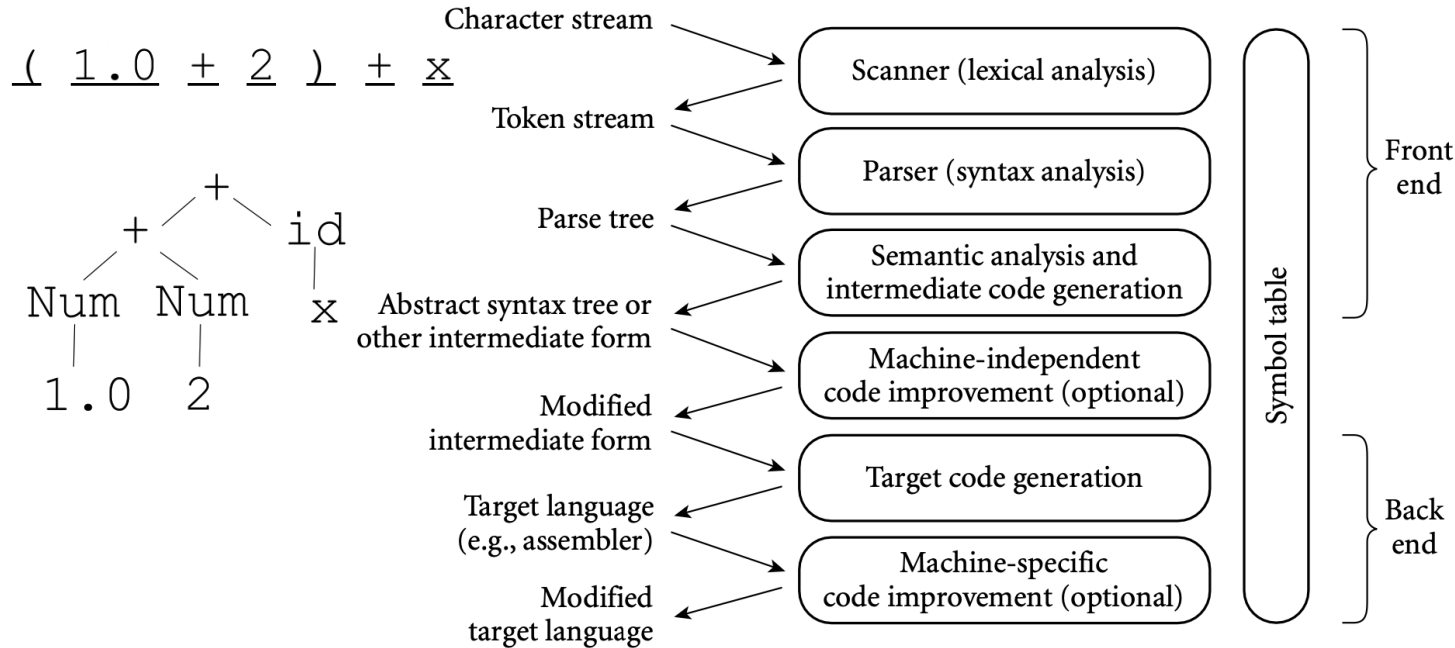
Compilation



Copyright © Elsevier Inc.

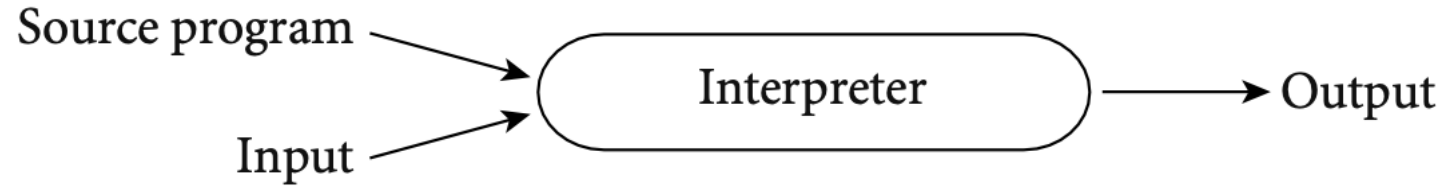
Compilation

$(1.0 + 2) + x$



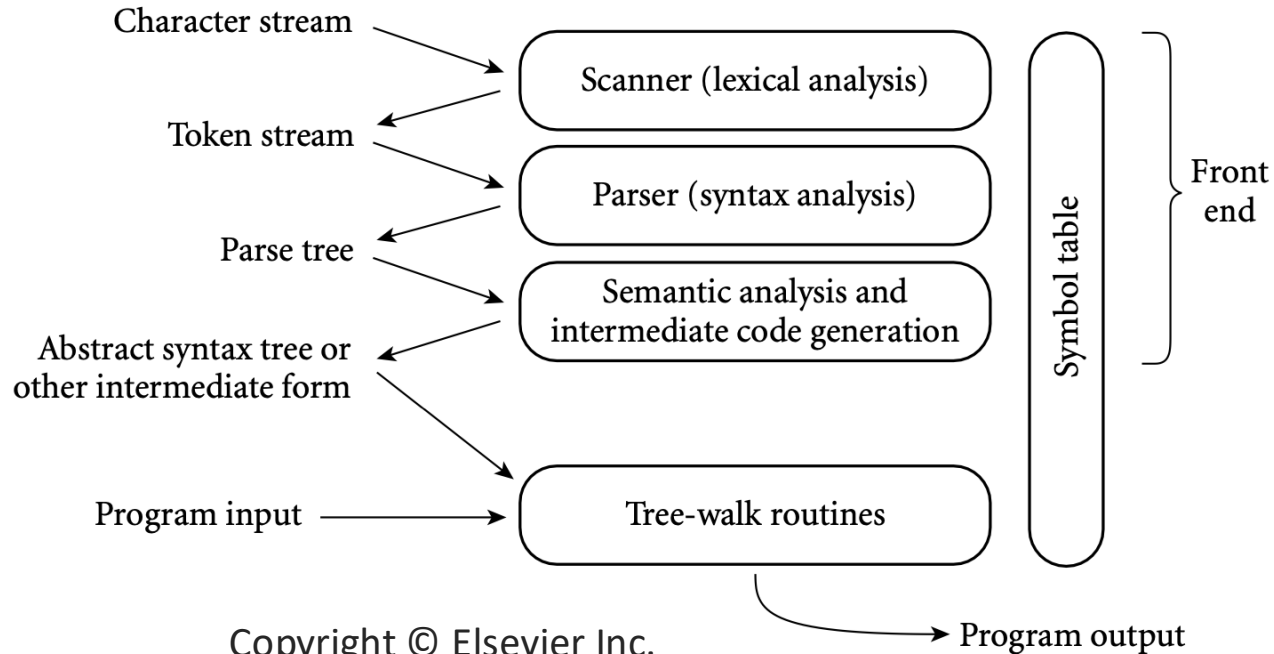
Copyright © Elsevier Inc.

Interpretation

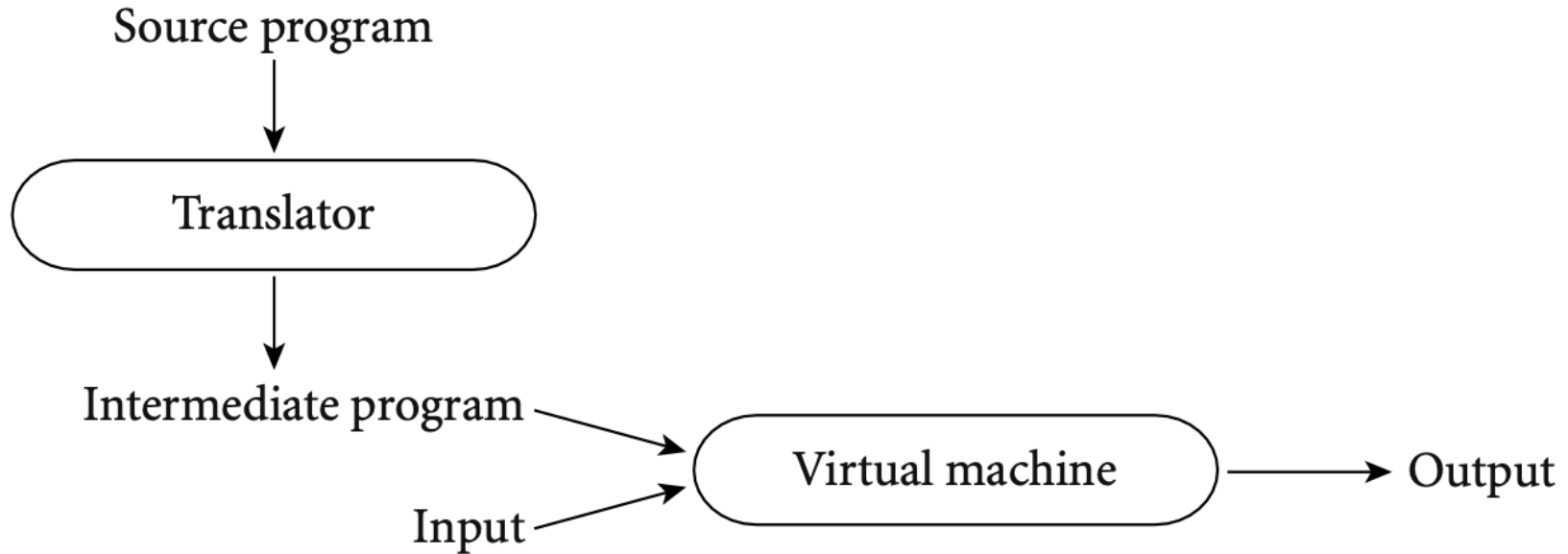


Copyright © Elsevier Inc.

Interpretation



A Mixture of Both



Example

Source code

```
int main() {  
    int i = getint(), j = getint();  
    while (i != j) {  
        if (i > j) i = i - j;  
        else j = j - i;  
    }  
    putint(i);  
}
```

Example

Scanner (Regular Expression)

```
int      main      (      )      {      int      i      =
getint   (      )      ,      j      =      getint   (
)      ;      while (      i      !=      j      )
{      if      (      i      >      j      )      i
=      i      -      j      ;      else      j      =
j      -      i      ;      }      putchar (      i
```

Example

Parser (Context-Free Grammar)

statement $\rightarrow$ *compound-statement*

compound-statement $\rightarrow$ { *block-item-list_opt* }

block-item-list_opt $\rightarrow$ *block-item-list*

block-item-list_opt $\rightarrow$ ϵ

block-item-list $\rightarrow$ *block-item*

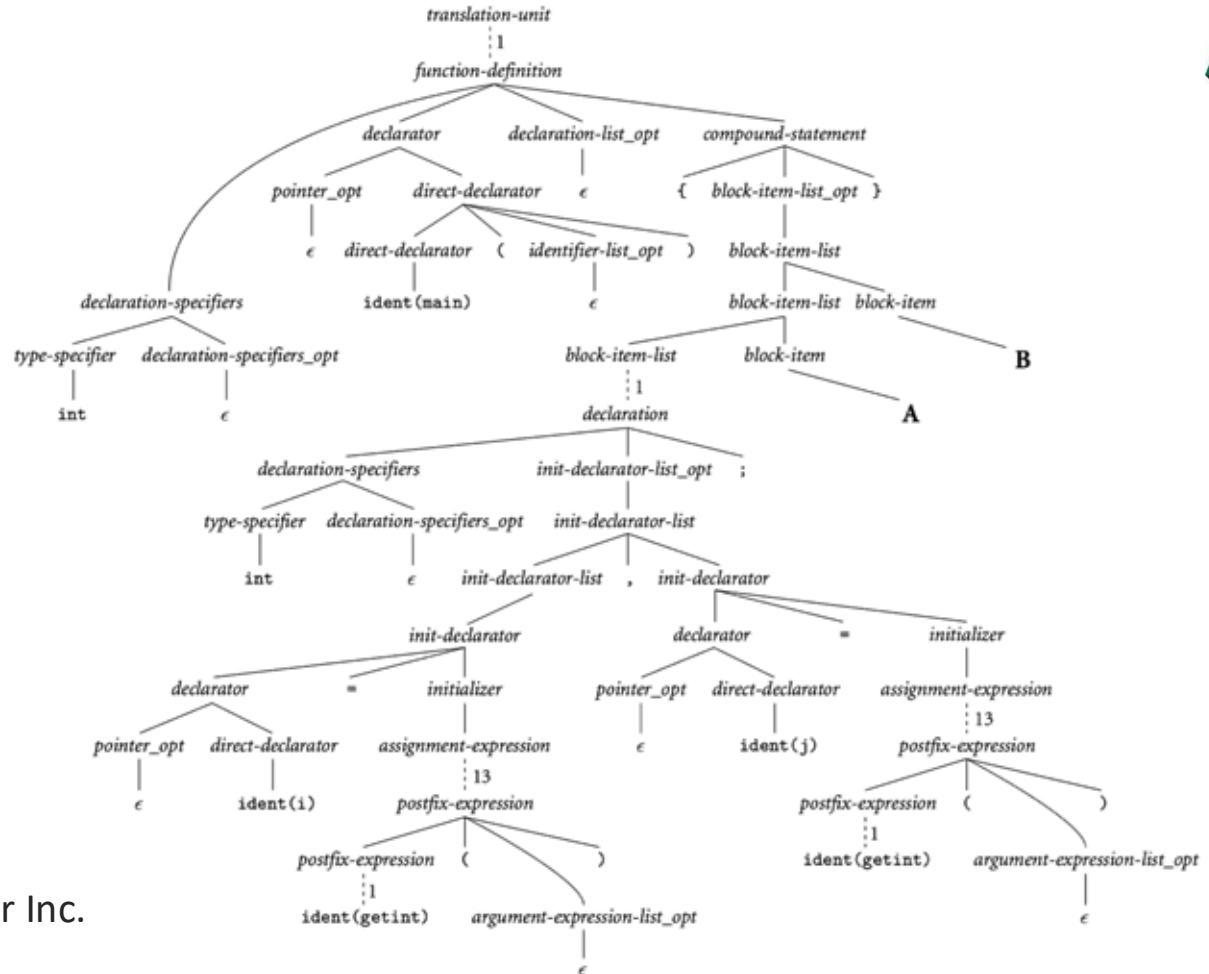
block-item-list $\rightarrow$ *block-item-list* *block-item*

block-item $\rightarrow$ *declaration*

block-item $\rightarrow$ *statement*

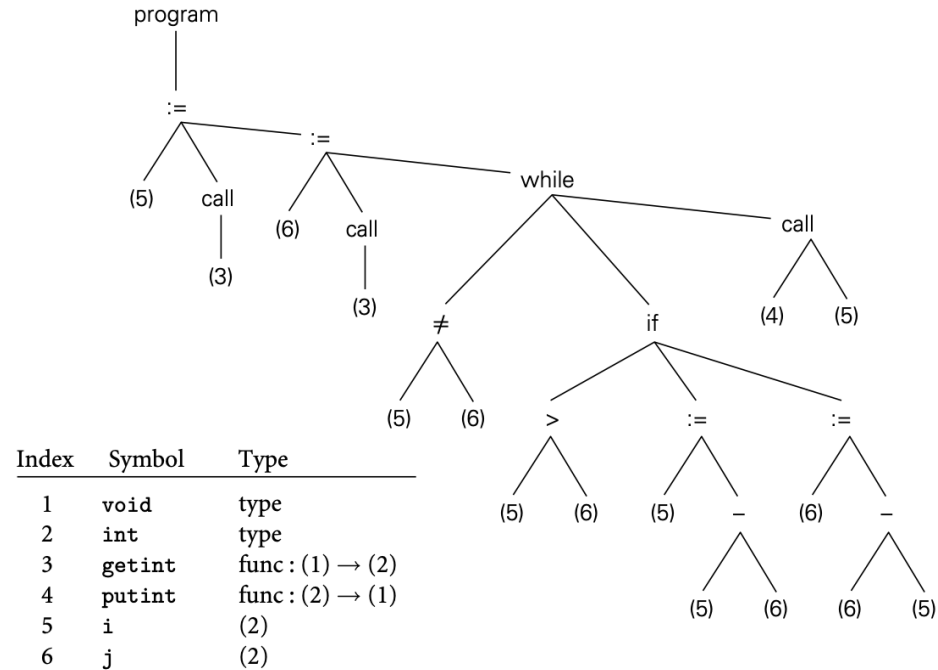
Example

Parse Tree



Example

Abstract Syntax Tree (AST)



Copyright © Elsevier Inc.

Why so many languages?

- People have different philosophies about how a program should be written
 - Endless debate about which language is the best
- Different app domains need different languages
 - Business domain: [Cobol](#)
 - Scientific computing: [Fortran](#), [C](#), [Python](#)
 - Systems programming: [C/C++](#)
 - Education: [BASIC](#), [Pascal](#), [Scheme](#), [Java](#), [Python](#)
 - Web: [JavaScript](#), [PHP](#)
 - Future domains: Cars, robots, Big-data/Parallel ...
- General-purpose vs. domain-specific languages
 - General-purpose languages: [Java](#), [C](#), [Scheme](#)
 - Domain-specific languages: [SQL](#), [Verilog](#) (for hardware design)

Popular Languages

Jan 2026	Jan 2025	Change	Programming Language	Ratings	Change
1	1		 Python	22.61%	-0.68%
2	4	▲	 C	10.99%	+2.13%
3	3		 Java	8.71%	-1.44%
4	2	▼	 C++	8.67%	-1.62%
5	5		 C#	7.39%	+2.94%
6	6		 JavaScript	3.03%	-1.17%
7	9	▲	 Visual Basic	2.41%	+0.04%
8	8		 SQL	2.27%	-0.14%
9	11	▲	 Delphi/Object Pascal	1.98%	+0.19%
10	18	▲	 R	1.82%	+0.81%

Source: [the TIOBE index](#)

Course Coverage

- Programming paradigms
 - Imperative, functional, logic, OOP
- Programming principles
 - Syntax
 - Semantics
 - Names, scopes, bindings
 - Types
 - Functions
 - Exception handling, memory management, etc.